Lab 03 Report: Real-Time LiDAR-Based Wall Following Method with Proportional-Derivative (PD) Control on a Physical Robotic Platform

Team 14

Connie Guo
Chenise Harper
Phillip Mason
Kareena Shah

6.4200

March 13, 2026

1 Introduction

Author(s): Kareena Shah

The contents of Lab 03 are divided into two primary objectives: wall following and object detection.

The wall follower consists of an autonomous controller that enables the racecar to drive forward while simultaneously maintaining a constant distance from the wall on either its left or right. The main purpose of the safety controller is to protect the physical system from crashes, especially for future labs when the racecar will be moving at high speeds. Both the wall follower and safety controller must be robust to account for environment changes and high speeds.

Our team further defined these two objectives with qualitative metrics.

1. Follow a wall at a desired distance, specified by our team to be 1 m, within a **$\pm$5% margin of error.**
2. Implement a safety controller that forces the car to stop **at least 0.10 m before an obstacle.**

These metrics of success were all agreed upon by our team and helped guide our data analysis and evaluation.

At a systems level, the wall follower and safety controller rely on Light Detection and Ranging (LiDAR) data and a Proportional-Derivative (PD) controller.

The racecar houses a LiDAR sensor at the front that takes spatial scans of its surroundings as seen in Figure 1. This laser data is then sliced and fed into a PD controller, a common control protocol that involves a proportional gain to reduce error and derivative gain to improve stability and dampen oscillations for optimal performance.

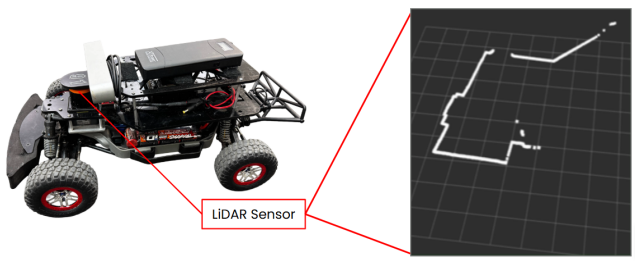


Figure 1: Our physical racecar features a LiDAR sensor that produces a laser scan as visualized to the right, and sends this to our PD controller.

This lab presents a new set of challenges as the performance of the wall follower in the simulation is not exactly one-to-one with its performance in the physical system. As a result, our team took an iterative approach to develop a robust wall follower and safety controller in real-life.

2 Technical Approach

Author(s): Chenise Harper

The objective of this lab was to create a robust autonomous navigation system consisting of a **Safety Controller** and a **Wall Follower** algorithm capable of keeping a racecar robot a set lateral distance away from walls and dynamically adjusting to avoid obstacles in its path.

2.1 Data Acquisition and Processing

The foundation of both algorithms is the processing of data from the robot's LiDAR sensor. This sensor calculates its distance to points in the environment

by emitting rapid light pulses at set angles and measuring the time they take to reflect back (1).

$$Distance = \frac{Speed\,of\,Light \cdot Time\,of\,Travel}{2} \tag{1}$$

2.1.1 Coordinate System

The raw data received from the LiDAR sensor is sent as *sensor_msgs/LaserScan* messages (See Appendix [5.1.1]) including a set of distances and their corresponding angles. This functions as a **Polar Coordinate** system (r, θ).

For easier spacial reasoning, this information was transformed into **Cartesian Coordinates** (x, y) relative to the */laser* frame (2):

$$x = r \cdot \cos(\theta), \quad y = r \cdot \sin(\theta) \tag{2}$$

In this frame, the robot is located at $(0, 0)$, with the x-axis pointing forward and the y-axis pointing to the left.

2.1.2 Angle Slicing

The LaserScan message receives many redundant angles. For processing efficiency, angular masks were used to 'focus' the robot's attention as seen in Figure 2:

1. **Front Mask:** A narrow window of ± 0.2 radians ($\approx 11.5°$) directly ahead of the robot is monitored.

2. **Side Mask:**

   - For the Safety Feature algorithm, the set of points between $45°$ and $115°$ (left) or between $-45°$ and $-115°$ (right) were used in the calculations.
   - For the Wall Follower algorithm, the set of points between $60°$ and $115°$ (left) or between $-60°$ and $-115°$ (right) were used in the calculations.

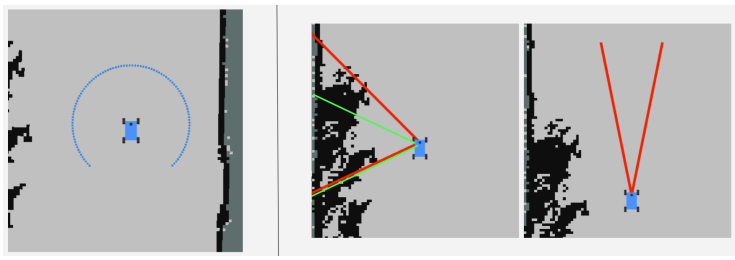


Figure 2: The left picture shows the full laser scan range. For conciseness, this scan is split into two masks. The safety side mask is 70° and the follower mask is 55°, pictured in the middle in red and green respectively. To the right, the front mask of ≈ 11.5° is shown. These masks are used in place of the full scan for modelling walls and obstacles.

2.2 Safety Controller

The safety controller is a high priority ROS2 node that acts as a fail-safe preventing crashes. It uses the *sensor_msgs/LaserScan* received through the */scan* topic to monitor the environment and override any drive commands if a collision is imminent.

2.2.1 Implementation

The controller evaluates stopping based on two conditions in **Polar Coordinates**:

1. **Static Proximity:** If any distance received in the **Front Mask** is closer than a set threshold (0.5 m).

2. **Dynamic Proximity:** The safe stopping distance buffer is based on the current velocity of the car ($0.5s \times V_{current}$).
   This speed is pulled from the *ackermann_msgs/AckermannDriveStamped* (See Appendix [5.1.2]) messages that wish to be published to the car through the */vesc/high_level/input/nav_0* topic (See Appendix [5.2]).

When a possible collision is detected, the node publishes an *AckermannDriveStamped* message to the */vesc/input/safety* topic (See Appendix [5.2]) with a speed of 0.0 m/s, effectively cutting power to the motors. The choice of the proximity values used in the algorithm takes into account environmental stimuli like forward momentum that affect the robot's stopping speed.

2.3 Wall Follower

The core navigation system in the Wall Follower algorithm relies on a PD control loop to regulate the car's steering angle.

2.3.1 Linear Wall Modeling

Possible environmental noise is reduced using linear regression. Firstly, the cartesian coordinates in the **Side Mask** are further sliced. Based on the robot's coordinate frame, the x-axis represents forward distance and the y-axis represents perpendicular distance. This means that outliers in the y-axis could affect the estimation of the followed wall. To remedy this, any point deviating more than 50% from the median y-value is rejected as an outlier before further calculation.

The median was chosen as the reference statistic because the mean is very sensitive to extreme values. If the LiDAR scans a smooth wall but one beam passes through a doorway or hits a far away object, that outlier will skew the average significantly.

To transform these discrete points into a navigable path, we use **Ordinary Least Squares (OLS)** regression via `np.polyfit`. This algorithm calculates the line of best fit by minimizing the sum of the squared vertical residuals (the distance between each LiDAR point and the calculated line). The resulting equation, $y = mx + c$, provides two critical pieces of information:

- **The Intercept (c):** The instantaneous lateral distance from the robot's center to the wall.
- **The Slope (m):** The angular orientation of the robot relative to the wall.

A slope of $m = 0$ would indicate that the robot is perfectly parallel to the wall while a positive or negative slope would imply that the car is at an angle converging or diverging from the wall.

A known edge case in linear regression is the "vertical line" problem. If the robot faces a wall head-on, the x-values have no spread, leading to an undefined or infinite slope. Our implementation includes a **Peak-to-Peak (ptp)** check using a built in numpy function `np.ptp(list_of_values)` If $x_{spread} < 0.1$m, the regression is considered unreliable and the system defaults to driving forward with a steering angle of 0.0.

2.3.2 Look-Ahead Wall Distance

At high speeds, reacting to the current distance causes the car to oscillate heavily. Using a **Look-Ahead** strategy, the car predicts where the wall will be at a distance L_{ahead} (where $L_{ahead} = V \cdot 0.5$ where $V = Velocity\ of\ the\ robot$). The predicted distance D_{pred} is calculated as:

$$D_{pred} = (m \cdot L) + c \tag{3}$$

The error at the robot's current location is therefore:

$$Error = D_{desired} - |D_{pred}| \tag{4}$$

2.3.3 The PD Law

We calculate the steering angle δ using the following feedback law:

$$\delta = -(K_p \cdot e(t) + K_d \cdot \frac{de(t)}{dt}) \cdot \text{SIDE} \tag{5}$$

The **Proportional (K_p)** term corrects the immediate error between D_{pred} and $D_{desired}$. The **Derivative (K_d)** term provides a damping effect by calculating the rate of change of the error ($\frac{e_t - e_{t-1}}{\Delta t}$), preventing overshooting as the car converges.

2.3.4 Frontal Detection Bias for Cornering

While the side-wall PD controller maintains lateral distance, it is inherently reactive to the wall currently parallel to the vehicle. To handle 90° corners and "dead-end" hallways effectively, we implemented a **Frontal Detection Error** bias. This logic serves as a predictive steering override by monitoring our **Front Angle Mask.**

The logic operates on three key principles:

1. **Velocity-Scaled Safety Buffer:** Rather than using a static distance, we define a *Safe Distance* (D_{safe}) that is dynamically scaled by the robot's current velocity:

   $$D_{safe} = D_{desired} \times V_{current} \tag{6}$$

   This ensures that at higher speeds, the robot begins its turn earlier to account for the physical turning radius and momentum of the chassis.

2. **Error Injection:** If the minimum distance in the front scan (D_{front}) falls below D_{safe}, we calculate a *Frontal Detection Error.* This value represents the"intrusion" into the safety buffer:

   $$e_{front} = (D_{safe} - D_{front}) \times 2.0 \tag{7}$$

   We apply a gain of 2.0 to this term to ensure that frontal obstacles take priority over side-wall tracking.

3. **Behavioral Blending:** This error is added directly to the standard PD error. This "tricks" the Proportional controller into perceiving a massive displacement from the side wall, triggering a maximum steering effort (saturated at 0.34 rad) before the side-wall regression itself would have signaled a turn.

2.4 The Output

Final steering commands are sent via the `ackermann_msgs/AckermannDriveStamped` format. This accounts for the physical constraints of a car-like robot, where the inner and outer wheels follow different radii. We constrain the output steering angle to ±0.34 radians to match the mechanical limits of the steering servo.

2.5 Summary of Parameters

The following table summarizes the constants used in the final implementation.

Parameter	Value	Description
K_p	3.0	Proportional Gain
K_d	$2.0 \times V$	Derivative Gain
L_{ahead}	$V \times 0.5$s	Look-ahead distance
δ_{max}	0.34 rad	Maximum Steering Angle

Table 1: System Control Parameters

3 Experimental Evaluation

Author(s): Connie Guo, Phillip Mason, Kareena Shah

3.1 Safety Controller

3.1.1 Test Setup

To evaluate the performance of our safety controller, we created a new node that allows the car to drive in a straight line. It continuously publishes a 0.0 angle steering command to the robot's navigation system, so the safety controller can override it when necessary. Our performance metric was based on the aforementioned safety controller objective of achieving a minimum 0.10 m distance between the racecar and an obstacle.

To validate the performance of our safety controller, we placed our racecar ∼1.5 m in front of an obstacle and drove it straight towards an obstacle at five different velocities, each with three trials for redundancy: 1 m/s, 1.5 m/s, 2 m/s, 2.5 m/s, and 3 m/s. At each trial run, we measured the distance between the front of the bumper and the obstacle, which was the red brick provided in our lab kits.

Figure 3: Our safety controller was assessed by driving head-on towards a brick at 5 speeds 3 times each and measuring the distance where the car stopped.

3.1.2 Results

The results depicted in Figure 4 present the individual stopping distances across three trials and average stopping distances over five velocities ranging from 1.0 m/s to 3.0 m/s. Although not all the average stopping distances meet the 0.10 m metric - specifically at 2.5 m/s - the car still stops before touching the obstacle, which we deem a success for our purposes.

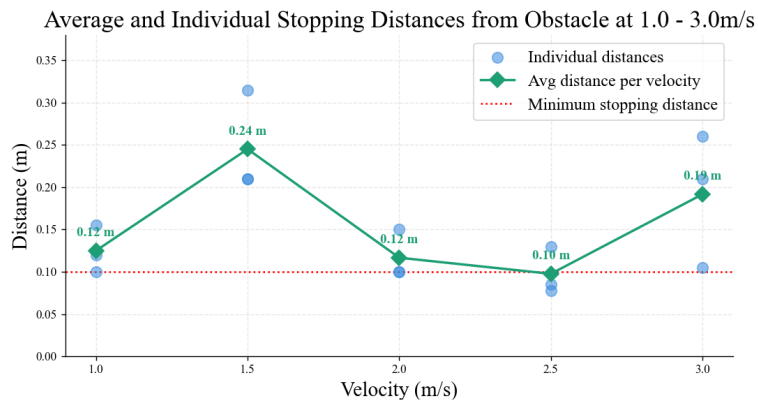


Figure 4: **Distribution of raw distance measurements across varying velocities (1.0–3.0 m/s).** Individual stopping distances per trial (blue circles) are plotted alongside the calculated average stopping distance per velocity (green diamonds). A majority of average stopping distances across the velocities have a minimum stopping distance of 0.10 m, which is our goal.

3.2 Wall Follower

Our wall follower was experimentally evaluated by conducting tests on a series of unique paths. The performance of the racecar was tested at two distinct velocities for each of these paths in order to quantitatively assess the robustness of our algorithm at different speeds and in varying environments.

3.2.1 Test Setup

We designed five unique test paths for wall follower testing, as seen in Figure 5. In each test, the car's initial position was 1 m (our desired distance) away from the wall. Each test case was run three times for redundancy for each velocity (1 m/s and 2 m/s). To quantify our results, the error between our desired distance and the true distance was averaged over the three trials for each speed, and compared to our performance metric of a $\pm 5\%$ margin of error. We define error as the difference between the car's measured distance from the wall and the desired distance of 1m from the wall.

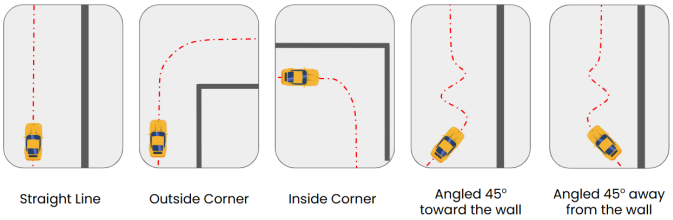


Figure 5: **We designed five wall follower test paths.** They included a straight line trajectory, a ∼90° inside corner, a ∼90° outside corner, and two straight paths with an initial angle offset of ±45°. Ensuring the car can follow these different paths supports our algorithm's robustness to handle untested paths it might encounter in real-world environments.

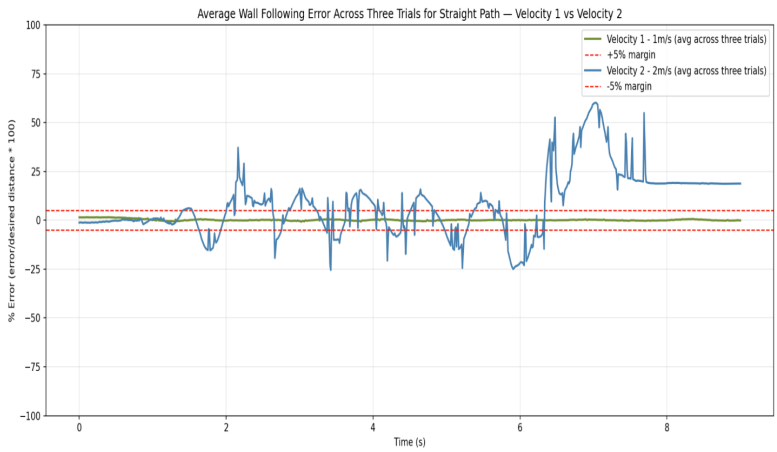
3.2.2 Results

Our five test cases tested a wide range of our wall follower capabilities and allowed us to assess our wall follower performance comprehensively. We use percent error to quantify our data.

$$\% \text{ Error} = \frac{D_{\text{measured}} - D_{\text{desired}}}{D_{\text{desired}}} \times 100 \tag{8}$$

Our first test case was a straight line approach to determine if our implementation would succeed under the most simple conditions.

Figure 6 shows the calculated percent error between the desired and actual wall distance on a straight line path averaged across three trials per velocity. At 1 m/s (green line), there is practically no oscillation for a straight line approach. The % error along the trajectory are all within the designated ±5% margin of error. However, at 2 m/s, the data shows uniform, clearly defined error oscillations that exceed our performance metric.



Wall Distance Error (%)	Velocity 1 (1 m/s)	Velocity 2 (2 m/s)
Minimum	-0.76	25.44
Maximum	1.55	48.78
Average	0.13	-2.08

Figure 6: **Our controller performs well at 1 m/s, but fails to converge at 2 m/s.** The percent error at 1 m/s (green) is well within the 5% margin of error (red). The percent error at 2 m/s (blue) oscillates outside our accepted range. The table summarizes the minimum, maximum, and average wall distance error for both velocities.

Our next set of tests placed the car at 45° towards or away from the wall. By starting our car with an initial known offset, we can analyze the response time and overshoot of its performance due to our controller gains. As seen in Figure 7, our error smoothly reaches a steady state within two seconds at 1 m/s, while at 2 m/s it crosses zero error within one second but continues to overshoot and steadily oscillates outside of our set margin.

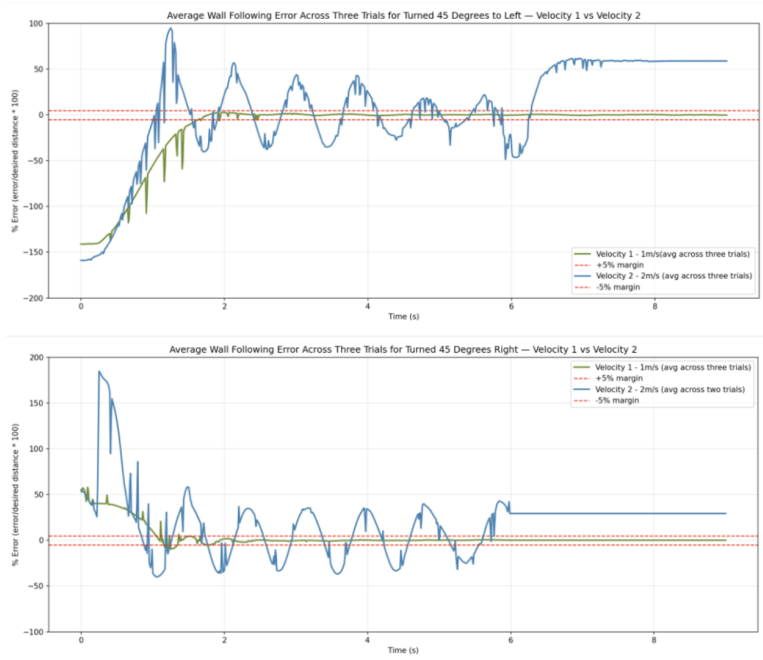


Figure 7: **Our controller gains are properly tuned for 1 m/s, but are underdamped at higher speeds.** At 2 m/s (blue), the performance has a faster response time but higher overshoot and oscillation. So, at higher velocity our controller likely has a proportional gain that is too high and a derivative gain that is too low.

Table 2: Max, Min, and Avg % Error when car rotated 45° to the left.

Wall Distance Error (%)	**Velocity 1 (1 m/s)**	**Velocity 2 (2 m/s)**
Minimum	-141.11	-159.01
Maximum	2.64	94.83
Average	-14.50	6.03

Table 3: Max, Min, and Avg % Error for car rotated 45° to the right.

Wall Distance Error (%)	**Velocity 1 (1 m/s)**	**Velocity 2 (2 m/s)**
Minimum	-9.43	-40.01
Maximum	58.01	184.77
Average	3.78	25.44

Of all tests conducted, the "turning corners" tests yielded the most variability.

As seen in Figure 8, there is a large overhead error when the car turns the corner. This is due to inaccuracies in the way the linear regression model maps the corner and also due to the physical pitch and roll of the car as it turns causing the LiDAR scans to not be on a perfectly parallel plane with the floor.

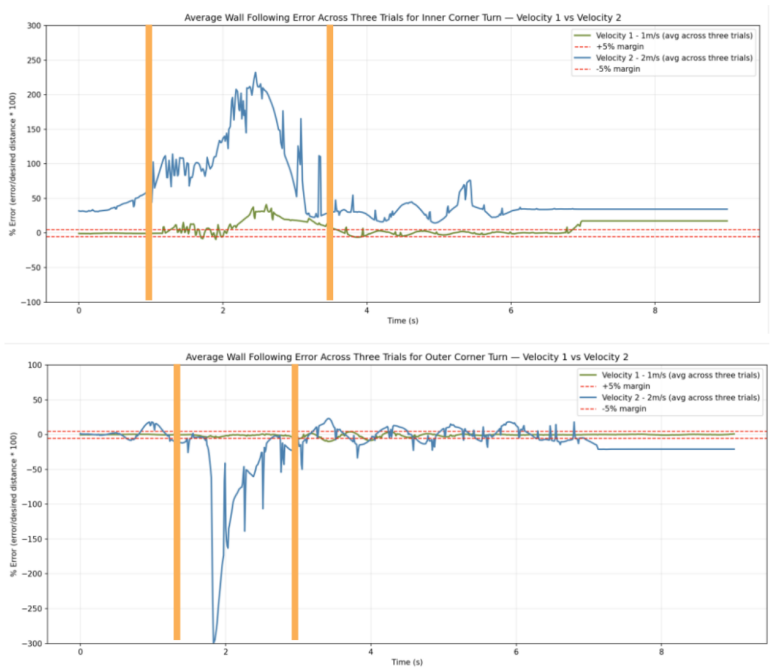


Figure 8: **During the turns, our controller experiences higher errors than in previous tests.** The region between the vertical orange lines signifies the portion of the trajectory at which the car was turning. At 2 m/s the error is about **3 times** higher than at 1 m/s.

Table 4: Max, Min, and Avg % of Error when car turns inner corner.

Wall Distance Error (%)	**Velocity 1 (1 m/s)**	**Velocity 2 (2 m/s)**
Minimum	-9.44	14.48
Maximum	62.96	232.21
Average	8.71	56.98

Table 5: Max, Min, and Avg % of Error when car turns outer corner.

Wall Distance Error (%)	**Velocity 1 (1 m/s)**	**Velocity 2 (2 m/s)**
Minimum	-9.84	-300.06
Maximum	4.12	23.01
Average	-0.77	-14.70

Clearly, we can see that at a baseline velocity of 1 m/s the physical implementation of the wall follower has optimal performance for all tested scenarios. However, beyond that velocity, the overall performance of the PD controller and physical system declines due to poor tracking and large oscillations that exceed our designated margin of error.

Test Case	**1 m/s Results**	**2 m/s Results**
Straight Line	**PASS**	**FAIL**
Angle Offset - Inward	**PASS**	**FAIL**
Angle Offset - Outward	**PASS**	**FAIL**
Inner Corner	**PASS**	**FAIL**
Outer Corner	**PASS**	**FAIL**

Table 6: **Summary of all test cases across both velocities.** At 1 m/s our controller passes all test cases within the ±5% margin of error. At 2 m/s, the wall follower performance is not optimal due errors exceeding our set margin.

4 Conclusion

Author(s): Phillip Mason, Kareena Shah

Overall, our wall follower and safety controller implementation succeeded in achieving the aforementioned objectives:

1. Follow a wall at a desired distance, specified by our team to be 1 m, within a **±5% margin of error.**
2. Implement a safety controller that forces the car to stop **at least 0.10 m before an obstacle.**

Visual observations showed that the racecar was able to successfully stop sufficiently far from an obstacle at different velocities. At lower velocities the static proximity stopped the car before it collided with objects, and at higher velocities the dynamic proximity stopped the car earlier to account for it rolling while decelerating. Through this joint detection method, the car was consistently

stopped before any collisions occurred, and without being too restrictive and inhibiting the wall follower.

The wall follower met the objectives set for it at 1 m/s, experiencing very low error between the desired distance and true distance from the wall. The results of all tests at different velocities had consistent performance across multiple trials as well. These consistent results in such varied conditions proved the robust nature of our wall follower and verified the accuracy of our implementation of slicing and linear regression to model the wall we were following. They also confirmed that our PD controller was properly tuned at 1 m/s. However, at 2 m/s the controller had vast oscillations with quicker response time and high overshoot, indicating that the system was underdamped. This implies that in addition to the derivative gain, the proportional gain should also depend on the car's velocity.

Future improvements include tuning both the proportional and derivative gains to vary based on velocity such that the system stays close to critically damped. The safety controller should also be tested mid-turn and in other situations where obstacles are not always stationary and faced head-on.

In conclusion, the safety controller performs reliably at varied speeds, while the current wall follower is robust to different environments but sensitive to different speeds.

4.1 Lessons Learned

4.1.1 Connie Guo

Throughout the course of lab 3, I have learned many things about myself and my collaboration abilities.

- To begin with, I was frustrated with my lack of technical ability to write sufficient technical reports, so I felt myself depending on my teammates a lot and asking for verification for every little thing that I wrote. And I felt as if I could not move on without confirming that a graph was sufficient or a sentence was appropriate for a technical report. However, I think the only way to improve is to continue writing more technical reports and perhaps reading papers.

- I have also learned that if I say I will do something, it is better to do it when I say I will instead of putting it off. This is because the sooner it is done, the more time an extra set of eyes can look over it. And, thus it will lead to less of my time and my teammates' time wasted. I have spent many a late night working on the project and not completing anything, so in order to prioritize my sleep and productivity, it's better to get as much as possible done during the day and preferably with my teammates.

4.1.2 Chenise Harper

Working on this project has brought me two main insights:

- The habit of "hard-coding" technical programs has worked for me so far but as I approach the more complicated problems brought up in this class, I have realized that a fundamental part of engineering is optimizing the testing process. I programmed a lot of the code used in this lab, but it was only deep within the testing process that I realized how ineffectively it was designed. I have learned that a more "robust" program may lack specificity but may often times be more useful for tuning or testing an algorithm.

- I have also learned that I can not do it all. This lab has been a very insightful journey into appreciating my strengths and recognizing my weaknesses. I am grateful to have gotten the group of Team 14 as I think that on a social and technical level, we mesh together quite well. I have learned how to rely on others for support while also trusting my teammates to reliably contribute to the lab.

4.1.3 Phillip Mason

I learned two important things from this lab that I think will remain important throughout both future labs and my career:

- This lab taught me the importance of testing our code early and frequently. I definitely overestimated how long one week of work was. Had we started testing earlier, we could have properly tuned our system's gains at different speeds. I was also the one who recorded the rosbags, and not knowing what metrics we might need, I ended up recording more topics then were ultimately necessary. Having a concise idea of how to assess each component as they are developed helps streamline both tuning and data visualization, and I hope to improve on this in the future.

- I have also learned the importance of debate and consensus. From the whole team initially deciding on how to consolidate our 4 wall followers to Chenise and I developing the safety controller and testing the wall follower, there is no element of this robot that I worked on entirely alone, and having those different viewpoints improved our car dramatically. Other people can catch things I miss, and having different ideas and making compromises to get the best parts of each improves both the end product and the experience of getting there.

4.1.4 Kareena Shah

This lab provided me with key insights on both the technical and team management levels:

- Working through this lab helped me realize the complex nature of test planning and data analysis. It was the first time I thought deeply about determining what data most accurately measured the properties of our system and best validated our implementation, all while ensuring our testing scheme was comprehensive to fit our data needs. I realized oftentimes answering these questions requires input from multiple parties to understand their perspectives and accommodate their needs.

- This experience also taught me a lot about time management and task allocation in a team setting. We parsed through the required deliverables right away and strategized our timeline to ensure that we met our deadlines through equally distributed work. While we did experience delays, I learned a lot about the importance of setting hard deadlines to increase accountability and I hope to implement these learnings into our future labs for a better workflow.

5 Appendix

5.1 Message Definitions

5.1.1 [1.1] sensor_msgs/LaserScan Compact Message Definition

std_msgs/msg/Header header
float angle_min
float angle_max
float angle_increment
float time_increment
float scan_time
float range_min
float range_max
float[] ranges
float[] intensities

5.1.2 [1.2] ackermann_msgs/AckermannDriveStamped Compact Message Definition

float32 steering_angle
float32 steering_angle_velocity
float32 speed
float32 acceleration
float32 jerk

5.2 [2] Node topic chart

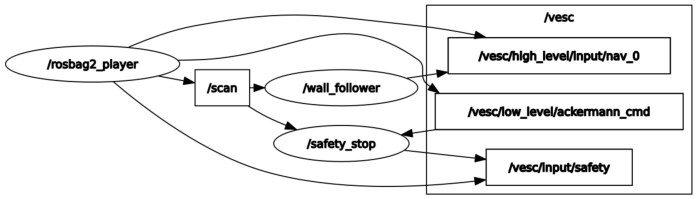


Figure 9: This is the publisher/subscriber tree of our algorithms using previously recorded data